alu-logo-white-full.png

FarmBridge — Direct Farmer-to-Buyer Agricultural Marketplace for Rwanda

Mugisha Moses (PM) Nkingi Gakindi Chris (Lead Dev) Hakizimana Chris Marcel (DB) Ineza Ghislaine (UI/UX) Kamy Uwambaye (Research)

African Leadership University | BSc (Hons) Software Engineering | Foundations Project | February – March 2026

1. Problem, Context & Users

Rwanda's 5.4M smallholder farmers lose 30–60% of crop value to middlemen — not because they produce less, but because they can't reach fair markets.

Agriculture drives **24% of Rwanda's GDP** and employs 70% of its workforce. Farmers (avg. 1 ha) are locked out of fair markets because:

- Intermediaries capture 30–60% of crop value
- Post-harvest losses reach 40% sub-Saharan Africa (FAO, 2022)
- Existing platforms (eHaho, Farmzone) are English-only, require internet, and have no in-app negotiation

Users	Smallholder farmers (Musanze, Nyagatare, Huye), verified buyers, admins
Gap	No Rwanda platform offers offline + Kinyarwanda + direct negotiation

2. Objectives & Scope

- **Aim:** Bilingual (EN/Kinyarwanda) PWA connecting farmers directly with verified buyers at zero fee.
- **Obj. 1:** Offline-first 3-tier architecture (Service Workers + IndexedDB).
- **Obj. 2:** Six modules — auth, listings, marketplace, real-time negotiation, orders, admin.
- **Obj. 3:** <3 s load, 90% task completion, ≥4/5 usability score.

In	PWA, listings, search, chat + price proposals, order lifecycle, Stripe + MoMo, ratings, notifications
Out	GPS tracking, hardware, mobile-money wallet management

3. Project Metadata & Team

Team	Moses (PM) · Chris N. (Dev) · Chris H. (DB) · Ghislaine (UX) · Kamy (Research)
Programme	BSc Software Engineering, ALU Rwanda, Cohort 2026
Duration	12-week sprint (Feb – Mar 2026)
Frontend	Next.js 16, React 19, Tailwind CSS, shadcn/ui
Backend	NestJS 11, Prisma ORM, Socket.io, BullMQ
Data	PostgreSQL (8 entities), Redis (cache + WS adapter)
External	Cloudinary, Firebase FCM, Stripe, Flutterwave

4. Constraints & Design Decisions

Constraint	Design Response
Rural connectivity	Offline-first PWA: Service Workers + IndexedDB background sync
Low digital literacy	Icon UI, EN↔Kinyarwanda toggle, onboarding walkthrough
No bank card	Flutterwave: M-Pesa, MTN Mobile Money, Orange Money
Data security	JWT access+refresh, bcrypt, Helmet, input sanitisation
Real-time chat	Socket.io + Redis adapter (horizontally scalable)
Image bandwidth	Cloudinary CDN: auto-resize, compression, lazy loading

Trade-off: Offline reliability prioritised over feature richness — core actions queue locally and sync on reconnect.

5. Proposed Solution & System Architecture

FarmBridge removes the middleman: farmers list → buyers negotiate directly → orders placed, paid, and tracked end-to-end.

User Flow

- Farmer lists products with photos & asking price
- Buyer filters marketplace by crop/location/price
- Real-time chat: text, voice notes, price proposals
- Deal agreed → Stripe card or Flutterwave MoMo
- Order tracked: PENDING → CONFIRMED → DELIVERED
- Both parties rate each other post-transaction

vs. eHaho / Farmzone

- Voice notes in negotiation chat
- Offline-first: list & browse without internet
- Verified profiles + mutual rating system

System Architecture

[Screenshot: screenshot landing]

6. Methodology & Build Process

Sprint 1	Research, ERD (8 tables), NestJS setup, JWT + Google OAuth
Sprint 2	Farmer listings + Cloudinary, buyer marketplace, search/filter
Sprint 3	Socket.io negotiation, price proposals, orders, Stripe + Flutterwave
Sprint 4	PWA offline, Firebase FCM, ratings, admin panel, integration tests

Stage	Key Output
Discover	10 personas; gap analysis of eHaho, Farmzone, MFarms
Design	ERD, Swagger API contract (16 modules), UML diagram
Build	43 Next.js pages, 16 NestJS modules, WebSocket gateway
Test	Unit tests, Swagger API tests, manual E2E checkout flow
Iterate	Voice notes + Flutterwave MoMo added post-persona review

7. Results, Evaluation & Demo Evidence

43	16	<3 s
Next.js pages built	NestJS modules	Target load time

Implemented modules

- **Auth:** JWT + Google OAuth, RBAC (FARMER/BUYER/ADMIN)
- **Marketplace:** full-text search, filters, Redis-cached featured
- **Negotiation:** WebSocket chat, voice notes, price proposals
- **Orders:** PENDING → CONFIRMED → DELIVERED → REFUNDED
- **Payments:** Stripe + Flutterwave M-Pesa/MTN/Orange
- **Notifications:** Firebase push for 11 event types

[Screenshot: screenshot marketplace]

Live demo (6 steps)

1. Register as farmer; upload product listing
2. Switch to buyer; search + filter marketplace
3. Open chat; send price counter-offer
4. Farmer accepts via proposal card
5. Buyer checks out (Stripe or MoMo)
6. Push notifications sent; both rate each other

[Screenshot: screenshot chat]

8. Impact, Contribution & Learning

- **Technical:** Production-grade PWA with event-driven negotiation engine (WebSocket + BullMQ), dual payment gateway, Firebase push for 11 events. Full TypeScript across 43 pages and 16 modules.
- **User impact:** Direct trade eliminates intermediaries capturing 30–60% of crop value. Increases farmer net income by **32%** (Nakasone et al., 2014; World Bank, 2021).
- **Research:** Confirmed offline-first design and Kinyarwanda support are the **top two adoption factors** for agri-platforms in sub-Saharan Africa (GSMA, 2022).
- **Alignment:** Directly advances **Rwanda Vision 2050** — commercialised agriculture, poverty reduction, youth tech employment.

Every middleman-free sale means the farmer keeps 30–60% more of their harvest's value.

[Screenshot: screenshot dashboard]

9. Limitations & Next Steps

Current limitations

- Offline Service Worker needs HTTPS production hosting — dev-tested, not yet deployed
- Flutterwave mobile money in sandbox; live credentials pending operator approval
- Kinyarwanda translation ≈70% complete (i18next wired up)
- No formal load/stress testing at scale

Immediate next steps

- Deploy Railway (NestJS) + Vercel (Next.js) with production secrets
- Complete Kinyarwanda strings; bilingual usability tests with 5 farmer personas
- Obtain Flutterwave live API keys; run real mobile money E2E test
- Add USSD fallback for feature-phone users

Long-term vision

- AI price recommendations from historical transactions
- Cooperative group-negotiation for bulk orders
- Irembo digital ID integration for instant KYC
- EAC expansion: Uganda, Tanzania, Kenya

10. References & Acknowledgements

References

- MINAGRI. (2018). *Rwanda PSTA4 Strategic Plan.*
- FAO. (2023). *Digital Agriculture Country Profile: Rwanda.*
- Nakasone, E., Torero, M., & Minten, B. (2014). The power of information. *Annual Review of Resource Economics, 6*(1), 533–550.
- GSMA. (2022). *Connected Society: Mobile Internet in Sub-Saharan Africa.*
- World Bank. (2021). *Reducing Post-Harvest Losses in Sub-Saharan Africa.*
- Wainaina, P. et al. (2020). Digital marketplace adoption in East Africa. *J. Agricultural Economics, 12*(3).

Acknowledgements

We thank ALU School of Science & Technology faculty for their guidance. Special recognition to Rwanda's smallholder farming community whose challenges inspired this work.

LaTeX TikZposter